

Proiect: DungeonMaster AI — RPG text-based single-player cu 2 agenți

Context

Construiesc un MVP pentru un proiect universitar (Management de Produs Software). Produsul este un joc RPG text-based single-player în stilul Dungeons & Dragons, unde un utilizator joacă o aventură narată complet de AI. Toată lumea, NPC-urile, povestea și consecințele acțiunilor sunt generate dinamic. Spre deosebire de un chatbot generic care „simulează” D&D, această aplicație folosește o arhitectură multi-agent cu memorie persistentă și reguli deterministe pentru zaruri.

Obiectiv

Generează un MVP funcțional care demonstrează colaborarea între 2 agenți AI specializați, cu state management corect și o interfață de chat playable.

Tech stack obligatoriu

- **Frontend:** Next.js 14 (App Router) + TypeScript + Tailwind CSS + shadcn/ui
- **Agenți:** Anthropic Claude API (model: `claude-sonnet-4-5`) cu tool use
- **State management:** Zustand pentru client-side, JSON file sau SQLite pentru persistență (alege ce e mai simplu)
- **Deploy target:** Vercel

Arhitectura cu 2 agenți

Agent 1: Dungeon Master (Narrator + Rules Engine)

Rol: Orchestrator principal. Primește input-ul utilizatorului, decide ce se întâmplă mecanic, narează rezultatul.

Responsabilități:

- Interpretează intenția jucătorului din text liber (ex: „atac goblinul cu sabia” → acțiune de combat)
- Decide dacă acțiunea necesită aruncare de zar și ce tip (Combat/Stealth/Persuasion/Perception/Athletics)
- Apelează tool-ul `rollDice(sides, modifier)` — NU simulează zaruri în text, folosește funcție deterministă
- Apelează tool-ul `updateGameState(changes)` pentru HP, inventar, locație, aur
- Cheamă Agent 2 (NPC Brain) când jucătorul interacționează cu un personaj non-player

- Narează rezultatul final într-un paragraf cinematografic (2-4 propoziții)

Tools disponibile:

- `rollDice(sides: number, modifier: number) → { roll, total, success }`
- `updateGameState(changes: Partial<GameState>) → GameState`
- `getNPCResponse(npcId: string, playerMessage: string) → string` (declanșează Agent 2)
- `getMemory(query: string) → string[]` (semantic search în istoric)

Agent 2: NPC Brain (Character Actor)

Rol: Joacă rolul personajelor non-player în vocea lor distinctă.

Responsabilități:

- Primește: NPC-ul vizat (cu personalitate + secrete), mesajul jucătorului, contextul relației
- Generează un răspuns IN CHARACTER (un pitic hangiu vorbește altfel decât un mag elf)
- Respectă personalitatea, secretele și ce ar ști acel NPC despre jucător
- Returnează DOAR ce spune NPC-ul (fără narațiune externă — aia o face DM-ul)

Format input pentru Agent 2:

```
json
{
  "npc": {
    "id": "innkeeper_borin",
    "name": "Borin",
    "race": "dwarf",
    "personality": "precaut, prietenos cu cei care îi câștigă încrederea",
    "secrets": ["știe locația comorii ascunse sub tavernă"],
    "speechStyle": "propoziții scurte, înjurături piticești ocazional"
  },
  "relationshipWithPlayer": "datorat — jucătorul i-a salvat berea de bandiți",
  "playerMessage": "Ai auzit ceva despre fiica primarului?",
  "recentContext": "jucătorul tocmai a intrat în tavernă"
}
```

State management — schema obligatorie

```
typescript
```

```

type GameState = {
  player: {
    name: string;
    class: 'warrior' | 'mage' | 'rogue';
    hp: { current: number; max: number };
    stats: { str: number; dex: number; int: number; cha: number };
    inventory: string[];
    gold: number;
    location: string;
  };
  world: {
    currentScene: string;
    discoveredLocations: string[];
    activeQuests: Quest[];
  };
  npcs: NPC[];
  shortTermMemory: Message[]; // ultimele 10 schimburi, intră în context
  longTermMemory: MemoryEntry[]; // evenimente importante (kill-uri, decizii majore,
};

```

REGULĂ CRITICĂ: HP, inventar, aur și stats NU se modifică niciodată prin text generat de LLM. Doar prin tool call-ul `updateGameState`. Astfel evităm halucinațiile de tipul „acum ai 50 de aur” când de fapt ai 12.

Conținut minim pentru MVP playable

- 1 personaj jucabil cu 3 clase la alegere (warrior/mage/rogue)
- 3 locații conectate: Taverna „Coroana Spartă”, Pădurea Lupului Negru, Peștera Goblinilor
- 4 NPC-uri: Borin (hangiu pitic), Lyra (vrăjitoare misterioasă), Captain Aldric (gardă a orașului), Grix (lider goblin)
- 1 quest principal: găsește flica dispărută a primarului
- Sistem de combat simplu: zaruri d20 + modifier vs. AC al inamicului
- Sistem de skill checks: d20 + modifier vs. dificultate (10/15/20)

UI cerut

Layout split:

- **Stânga (70%):** zona de chat. Mesajele DM-ului în stil narativ (italic + culoare warm), mesajele jucătorului aliniate dreapta, rezultatele zarurilor evidențiate cu badge (ex: „ Persuasion: 17 — Succes”)

- **Dreapta (30%):** sidebar cu stats — HP bar, inventar, aur, locația curentă, quest activ
- **Jos:** input text cu placeholder „Ce faci?”

Folosește shadcn/ui pentru componente (Card, Badge, Progress, ScrollArea).

Structura fișierelor cerută

```
/app
  /api
    /game
      route.ts          # endpoint principal, primește input jucător
    /agents
      dungeonMaster.ts  # Agent 1
      npcBrain.ts       # Agent 2
  /game
    page.tsx           # ecranul de joc
  /character
    page.tsx           # creare personaj
  /lib
    /tools
      dice.ts           # rollDice deterministă
      gameState.ts      # updateGameState, getState
      memory.ts         # short/long term memory
    /data
      npcs.json         # fișele NPC-urilor
      locations.json    # descrierile locațiilor
      agentClient.ts    # wrapper peste Anthropic SDK
  /components
    ChatMessage.tsx
    StatsSidebar.tsx
    DiceRoll.tsx
  /store
    gameStore.ts        # Zustand
```

Flow exemplu pentru un turn complet

1. Jucător scrie: „Mă duc la hangiu și îl întreb de fiica primarului”
2. POST `/api/game` cu mesajul + state curent
3. Agent 1 (DM) primește input, identifică: e o conversație cu NPC + skill check de Persuasion
4. Agent 1 apelează `rollDice(20, +3)` → 17, succes
5. Agent 1 apelează `getNPCResponse("innkeeper_borin", ...)` → declanșează Agent 2

6. Agent 2 generează răspunsul lui Borin în character
7. Agent 1 primește răspunsul, îl integrează în narațiune cinematografică
8. Agent 1 apelează `updateGameState` dacă e cazul (ex: deblochează locația „Pădurea Lupului Negru”)
9. Răspuns final trimis la frontend: narațiune + dialog NPC + roll badge + state actualizat

Ce să livrezi

1. Structura completă de fișiere de mai sus
2. Cod funcțional pentru cei 2 agenți cu tool use real (nu mock-uri)
3. UI playable cu cel puțin 5-10 minute de gameplay posibil
4. README cu instrucțiuni de setup (env vars pentru `ANTHROPIC_API_KEY`)
5. Cel puțin un exemplu de conversație completă în README ca demo

Constrângeri

- Nu folosi LangChain/LangGraph — vreau control direct peste prompt-uri și tool use
- Nu salva în localStorage (cerință de la platformă) — folosește Zustand în memorie + opțional un endpoint `/api/save` pentru persistență server-side
- Răspunsurile DM-ului: maxim 4 propoziții pentru a păstra ritmul de joc
- Toate prompt-urile sistem ale agenților să fie în fișiere separate, ușor de modificat (`/lib/prompts/dmPrompt.ts`, `/lib/prompts/npcPrompt.ts`)

Începe prin a-mi arăta structura de fișiere și prompt-urile sistem pentru cei 2 agenți, apoi implementează endpoint-ul `/api/game` cu tool use funcțional. La final, UI-ul.